

FIXED-POINT DSP LOW PASS FILTER

overview

This project demonstrates how to run a **digital low-pass filter on an 8-bit microcontroller** that does not have a floating-point unit. Instead of using slow floating-point math, we implement the filter using **fixed-point arithmetic (Q15 format)**. Input audio samples are scaled into the Q15 range, the filter computation is carried out using integer math with bit-shifts for efficiency, and the output is then scaled back to human-readable form. By doing this, we achieve fast filtering while conserving both CPU cycles and memory.

The input data for testing comes from a **simulated “fake audio” signal**, represented as a small array of values ranging from 0 to 100. These values mimic varying amplitudes in an audio waveform. Each sample is first normalized and converted into Q15, then passed through the filter equation:

$$y[n] = b_0x[n] + b_1x[n-1] + a_1y[n-1]$$

This produces an output that smooths the signal by reducing rapid changes, exactly what a low-pass filter should do. The results are printed to the Serial Monitor for analysis.

This experiment highlights the **practicality of fixed-point DSP** on resource-limited microcontrollers. Even though the Arduino Uno’s processor is an 8-bit ATmega328P, it can still handle 16-bit fixed-point math efficiently thanks to its hardware multiplier. Using Q15 preserves enough precision for meaningful filtering, while avoiding the overhead of floating-point math. This approach can be applied to real audio or sensor signals, demonstrating how low-power devices can perform DSP tasks in applications like voice processing, signal smoothing, or noise reduction.

HOW I DID IT

basically i used fixed point arithmetic as a restriction of no fpu is given so multiply by 2^{15} and do non floating point arithmetic and then divide to get the same value as before so here we basicallly just use normal arithmetic in the given code i filter out about 75 percent of the frequency .

I got the equation bilinear transformation of a rc low pass filter of first degree

rather than using a direct audio signal i used a array for its values to make processing easier

SOURCE CODE

```
#include <Arduino.h>
```

```
#include <stdint.h>
```

```
#define TO_Q15(x) ((int16_t)((x) * 32768.0f))
```

```
#define TO_FLOAT(x) ((float)(x) / 32768.0f)
```

```
float fake_audio_float[16] = { 50.0, 69.1, 85.4, 96.2, 100.0, 96.2, 85.4, 69.1,  
                               50.0, 30.9, 14.6, 3.8, 0.0, 3.8, 14.6, 30.9 };
```

```
int16_t fake_audio_q15[16];
```

```
int16_t prev_input = 0;
```

```
int16_t prev_output = 0;
```

```
int16_t coef_b0 = TO_Q15(0.2929f);
```

```
int16_t coef_b1 = TO_Q15(0.2929f);
```

```
int16_t coef_a1 = TO_Q15(-0.4142f);
```

```
int16_t lowpass_filter(int16_t input) {
```

```
    int32_t acc = 0;
```

```
    acc += ((int32_t)coef_b0 * input) >> 15;
```

```
    acc += ((int32_t)coef_b1 * prev_input) >> 15;
```

```
    acc += ((int32_t)coef_a1 * prev_output) >> 15;
```

```
    int16_t output = (int16_t)acc;
```

```
    prev_input = input;
```

```
    prev_output = output;
```

```
    return output;
```

```
}
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    for (int i = 0; i < 16; i++) {
```

```
        fake_audio_q15[i] = TO_Q15(fake_audio_float[i] / 100.0f);
```

```
    }
```

```
    for (int i = 0; i < 16; i++) {
```

```
        int16_t in_q15 = fake_audio_q15[i];
```

```
        int16_t out_q15 = lowpass_filter(in_q15);
```

```
        float in_print = TO_FLOAT(in_q15) * 100.0f;
```

```
        float out_print = TO_FLOAT(out_q15) * 100.0f;
```

```
        Serial.print("Input: ");
```

```
        Serial.print(in_print, 2);
```

```
        Serial.print("\tFiltered: ");
```

```
        Serial.println(out_print, 2);
```

```
    }
```

```
}
```

```
void loop() {}
```

OUTPUT

